Support Vector Machines for Face Recognition using `scikit-learn`

CS 280 Mini Project
Joshua Frankie B. Rayo
2011-19612

November 29, 2016

Contents

List of Figures

Listings

1 Introduction

Image classification, particularly face recognition is one of the important applications of using intelligent systems. Facial images of different people are used to train a model

hoping to recognize familiar faces in the future. Supervised learning is possible using artificial neural networks (ANN) and support vector machines (SVM), while unsupervised learning is possible using eigenfaces from principal component analysis.

ANN implements the empirical risk minimization pprinciple that seeks to minimize the misclassification error of the training data, while SVM implements the structural risk minimization principle the seeks to minimize an uppper bound of generalization error. The solution of SVM may be global optimum while ANN may tend to fall into a local optimal solution. Consequently, overfitting is likely to occur with neural networks. [3]

Backproragation neural networks suffers from difficulty in optimizing large set of parameters including number of hidden layers, nodes per hidden layer, activation function, weights, learning rate and momentum rate. [3] On the other hand, there are fewer parameters to tune, namely: kernel function, hyperparameters and C for the non-separable cases in linear SVM. [4]

Performance of ANN and SVM are always compared with respect to solving specific classification problems. Each has its own advantages and disadvantages. Plenty of softwares already implemented these models together with other classification algorithms.

2 Problem Statement

In this research the optimal hyperparameters of SVM will be determined with respect to the face recognition problem. People's faces will serve as training data and faces with sunglasses are for testing. The best classifier should recognize the person behind an altered face correctly in high accuracy.

In this project, supervised learning is employed to make a model suitable for face recognition. Specifically, generalization performance of SVM will be discussed for face recognition activity. Model training and validation is done using Python `scikit-learn` package.

3 Methodology

3.1 Dataset

Training and validation images are obtained from *CMU face images data set*. Each subdirectory contains facial images of a specific person with different poses, with and without sunglasses. There are total of 20 persons to be classified with 16 poses each. Sample images can be seen at figures 1 and 2 below.

Figure 1: Different poses of a male person, with and without sunglasses.

Figure 2: Different poses of a male person, with and without sunglasses.

The images obey the PGM (portable gray map) format specifications [2].

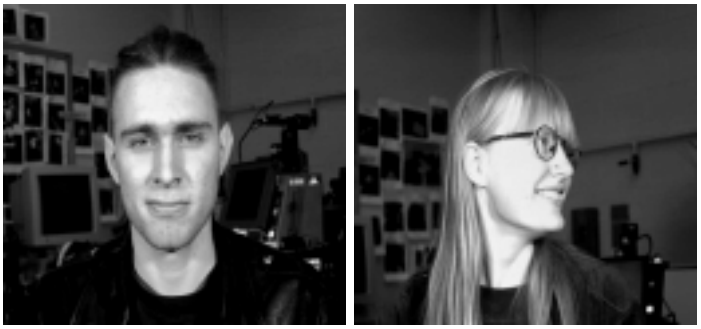

Figure 3: Sample PGM images of a smiling male and a smiling female with sunglasses, side view.

When a PGM image is opened in a plain text editor similar text will be as follows:

```
   P2
24 7
15
0 0 0 0 0 0 0 0 ...
0 3 3 3 3 0 0 7 ...
0 3 0 0 0 0 0 7 ...
0 3 3 3 0 0 0 7 ...
0 3 0 0 0 0 0 7 ...
0 3 0 0 0 0 0 7 ...
0 0 0 0 0 0 0 0 ...
```

The first three rows of the image are just metadata. Each image are of size 128 * 160 pixels for a total of 15360 attributes and the feature vector is formed row-wise. There are 320 training images (faces without sunglasses) and 320 validation images (faces with sunglasses). After forming the labels and input vectors the ANN and SVM are now ready to train.

3.2 The Classifiers

The code is made using Python, a high level general-purpose programming language that is relatively easy to learn and has very good features in list manipulation. The `sklearn` library of Python was used. Other libraries used are `numpy` and `scipy`.

We have obvious reasons of using existing libraries. More time is dedicated for training the dataset and tuning the parameters instead of implementing the classifier. Source codes released in the open-source community might be better than our own implementation. Continuous contribution of the community through GitHub make `scikit-learn` library compatitive for machine learning.

In this project `scikit-learn` will be used for face recognition activity, the most popular machine learning library for Python. [5] It contains simple and efficient tools

for data mining and data analysis including classification, regression, clustering, dimensionality reduction, model selection and data preprocessing.

Documentation of `scikit-learn` for ANN and SVM can be found at [5]. The method signature of `sklearn.svm.SVC` is as follows:

```
class sklearn.svm.SVC(C=1.0, kernel='rbf', degree=3, gamma='auto', coef0=0.0,
shrinking=True, probability=False, tol=0.001, cache_size=200, class_weight=None,
verbose=False, max_iter=-1, decision_function_shape=None, random_state=None)
```

It is a C-Support Vector Classification based on `libsvm`, where the multiclass support is handled according to a one-vs-one scheme. Linear, polynomial, RBF, sigmoid and precomputed kernels are supported. Relevant attributes and methods used for this research are the following:

- `fit(X, y)`: fit the SVM model according to the given training data
- `predict(X)`: perform classification on samples in input data X
- `score(X, y)`: returns the mean accuracy on the given test data and labels
- `support_vectors_`: returns the list of support vectors of the SVM model
- `n_support_`: returns the number of support vectors for each class

The parameter `gamma` measures how far the influence of a single instance reaches in the multi-dimensional space. Low values mean 'far' and high values meaning 'close'. This can be seen as the inverse of the radius of influence of each instances selected by the model as support vectors [7]. The `C` parameter trades off misclassification against simplicity of the decision surface. A low `C` makes it smooth while high `C` aims at classifying training instances correctly by giving the model freedom to select more samples as support vectors [7].

To estimate optimal parameters for the SVM classifier, the `GridSearchCV` class is used with the following signature:

```
class sklearn.model_selection.GridSearchCV(estimator, param_grid, scoring=None,
fit_params=None, n_jobs=1, iid=True, refit=True, cv=None, verbose=0,
pre_dispatch='2*n_jobs', error_score='raise', return_train_score=True)
```

This class utilizes exhaustive search over specified parameter values for a specific classifier. The parameters of a classifier are optimized by cross-validated grid-search over a parameter grid. Possible parameters are assigned in the `param_grid` variable. Initialization of this class is similar below.

```
tuned_parameters = ['kernel': ['rbf'], 'gamma': ['auto'] +
np.ndarray.tolist(gammas), 'C': C, 'kernel': ['linear'], 'C': C, 'kernel':
['poly'], 'C': C, 'degree': poly_d, 'gamma': ['auto'] +
np.ndarray.tolist(gammas), 'coef0': coes, 'kernel': ['sigmoid'], 'C': C,
'gamma': ['auto'] + np.ndarray.tolist(gammas),
'coef0': coes]
clf = GridSearchCV(svm(), tuned_parameters, cv=4, n_jobs=-1, verbose=10)
clf.fit(X, y)
```

Different parameters for SVM are obtained such that it maximizes testing accuracy. Accuracy and running time are performance measures used in comparison. Support vectors will also be extracted from the best SVM model to get the important faces from the dataset.

4 Results and Discussion

For a fixed kernel but different parameters, the accuracy of classification changes. The table plot the accuracy of SVM given several hyperparameters. These are some of the results `GridSearchCV` displayed in optimizing the SVM.

Kernel	**Param**	**Accuracy** (%)
linear		88.7500
polynomial	d=1	88.7500
	d=2	88.7500
	d=3	89.6875
	d=4	90.0000
	d=5	90.6250
	d=6	89.3750
	d=7	85.6250
	d=8	83.1250
	d=9	80.9375
	d=10	77.8125
RBF	g=1/(128*120)	29.0625
sigmoid	g=1/(128*120)	5.0000

Using a polynomial kernel makes the SVM better in generalization. Varying image background (see figures 1 & 2) is one of the reasons why this classifier did not achieved full accuracy.

It is observed that accuracy is not sensitive to `gamma`. It is also worth noting that there are 291/320 support vectors from the input data, which means that most of these are support vectors and of similar importance in training the SVM. In fact, each input instance contains unique facial expressions from each person. This is an expected result. Training and testing each SVM instance takes less than a minute.

It is a good idea to treat each pixel as a feature as suggested by literature because each person has different facial features like skin color, shape, etc. Even if one uses sunglasses or change its facial expression, the SVM can nearly recognize the face of that person. For the sigmoid function appears to the worst kernel whatever γ value is used. The polynomial function is the best SVM kernel for face recognition.

5 Conclusion

It is established that SVM performs better than ANN for the image classification problem involving supervised learning. Another competitor of SVM comes from a class of unsu-

pervised learning methods, the Principal Component Analysis specifically the eigenfaces algorithm [6]. One can also look for deep learning to solve this problem [8].

References

[1] UCI Machine Learning Repository, *CMU Face Images Data Set.* https://archive.ics.uci.edu/ml/datasets/CMU+Face+Images

[2] J. Poskanzer, *PGM Format Specification.* http://netpbm.sourceforge.net/doc/pgm.html

[3] K. Kim. *Financial time series forecasting using support vector machines.* Dongguk University, 2003.

[4] Z. Huang, et. al. *Credit rating analysis with support vector machines and neural networks: a market comparative study.* University of Arizona, 2003.

[5] scikit-learn, Machine Learning in Python. http://scikit-learn.org/stable/index.html.

[6] Eigenfaces for Face Recognition http://www.vision.jhu.edu/teaching/vision08/Handouts/case_study_pca1.pdf

[7] Plot different SVM classifiers in the iris dataset. http://scikit-learn.org/stable/auto_examples/svm/plot_iris.html#sphx-glr-auto-examples-svm-plot-iris-py

[8] Y. Taigman, M. Yang, M. Ranzato, W. Wolf. *DeepFace: Closing the Gap to Human-Level Performance in Face Verification.* https://research.facebook.com/publications/deepface-closing-the-gap-to-human-level-performance-in-face-verification/